

CS4051 – Information Retrieval

Chapter 21: Link Analysis

Week 15 | Muhammad Rafi | May 04, 2026

Complete Study Notes — Slides + Book + Q&A + Numericals

PART 1: Introduction & PageRank

1. What is Link Analysis? (The Big Picture)

When you search something on Google, Google must decide which page is most important. One powerful way is to look at who is linking to whom.

Core Idea: If many pages are pointing/linking to a page, that page is probably important.

This is borrowed from academic citation analysis — if many papers cite your paper, your paper is considered influential. The web works the same way. A hyperlink from one page to another is like saying 'this page is worth visiting.'

Problem: Simply Counting Links

Someone could create 100 fake websites all pointing to their page to boost their count. This is called **link spam**. So we need something smarter — that is where PageRank comes in.

2. The Web as a Graph

Before PageRank, we need to understand how the web is structured:

- Every **webpage** is a **node**
- Every **hyperlink** is a **directed edge (arrow)** from one page to another

So if Page A has a link to Page B, there is an arrow from $A \rightarrow B$.

Two Assumptions (From Slides)

1. The **anchor text** pointing to page B is a good description of page B.
2. A **hyperlink from A to B** represents an endorsement of page B by the creator of page A. (Not always true — internal navigation links like copyright notices do not really count as endorsements.)

Why Anchor Text Matters

IBM's homepage may not contain the word 'computer', but thousands of links pointing to ibm.com use 'computer' in their anchor text. Search engines use this as a powerful signal. This motivates why hyperlinks matter at all.

3. The Random Surfer Model

Imagine a person randomly browsing the web:

- They land on some page
- They randomly click any link on that page
- They go to that page, then randomly click again
- They keep doing this forever

Key Question PageRank asks: After a long time of random surfing, how often does this person end up on each page?

Pages visited more frequently = more important = higher PageRank.

4. Damping Factor & Teleporting

The Problem

What if the surfer reaches a page with no outgoing links (a dead end)? They are stuck. Also, what if there is a loop between two pages endlessly pointing to each other? The surfer gets trapped forever.

The Solution: Teleporting

Instead of getting stuck, the surfer can at any point **jump to a completely random page** on the web — like typing a random URL and starting fresh.

The Damping Factor (d)

This controls how often the surfer teleports vs follows a link:

- With probability **d** → surfer **follows a link** normally
- With probability **(1-d)** → surfer **teleports** to a random page

In practice, **d = 0.85** is commonly used (85% follow link, 15% teleport).

This is why the PageRank formula has two parts:

$$PR(P) = (1-d) + d \times (\text{sum of contributions from linking pages})$$

- **(1-d)** = the teleportation part
- **d × (...)** = the normal link-following part

5. Markov Chain & The Matrix

What is a Markov Chain?

At every step, where you go next depends **ONLY** on where you are right now. The surfer does not remember where they have been — only the current page matters. This is the **Markov Property**.

Turning the Web into a Matrix

We represent the entire web graph as a matrix where:

- Each **row** = a page you are coming FROM
- Each **column** = a page you are going TO
- Each **value** = probability of moving from one page to another

Example: If Page 1 has 2 outgoing links (to Page 2 and Page 3):

- Probability of going to Page 2 = 1/2
- Probability of going to Page 3 = 1/2

Each column adds up to 1 (total probability = 100%).

The Eigenvector

When you multiply this matrix by the PageRank vector repeatedly, it eventually stops changing. That final stable vector = the **PageRank scores**. This method is called **Power Iteration**.

6. PageRank Algorithm Symbols

Symbol	Meaning
P	A web page
d	Damping factor — probability that a user follows a link (not teleport)
PR(P)	PageRank of page P
deg(P)–	Number of links coming INTO page P (in-degree)
deg(P)+	Number of links going OUT of page P (out-degree)
N(P)–	Set of pages that point TO P (in-neighborhood)
N(P)+	Set of pages P points TO (out-neighborhood)
W	Hyperlink matrix representing the network
x	Eigenvector containing ranks for each vertex

7. PageRank Formula

Basic Formula:

$$PR(P_n) = \text{SUM of } [PR(P_i) / \text{deg}(P_i)^+]$$

Full Formula with Damping Factor:

$$PR(P) = (1-d) + d \times (PR(P_1)/\text{deg}(P_1)^+ + PR(P_2)/\text{deg}(P_2)^+ + \dots + PR(P_n)/\text{deg}(P_n)^+)$$

8. PageRank Algorithm Steps (Pseudocode Explained)

Step 1 — Initialize: Every page starts with $PR = 1/N$ (equal probability). If 4 pages exist, every page starts with $PR = 0.25$.

Step 2 — Iterate: Every page passes its PageRank to the pages it links to, divided equally among all its outlinks.

Step 3 — Apply Damping: $PR(P_i) = d/N + (1-d) \times (\text{collected contributions})$

Step 4 — Normalize: Make sure all PR values add up to 1.

Step 5 — Repeat Until Convergence: Keep repeating Steps 2–4 until values stop changing. Those final values = the PageRank scores.

Q: Does the pseudocode follow the same method as the matrix example?

A: YES. Pseudocode says initialize $PR=1/N \rightarrow$ we started with $[0.25, 0.25, 0.25, 0.25]$. Pseudocode says iterate until convergence $\rightarrow$ we kept multiplying $A \times v$. Pseudocode says normalize $\rightarrow$ values always sum to 1. Pseudocode says stop when values don't change $\rightarrow$ we stopped at iteration 7–8. Same thing, just pseudocode shows it in programming style and the example shows it numerically.

Q: What is this iterative matrix multiplication method called?

A: It is called Power Iteration. Repeated matrix-vector multiplication converges to the principal eigenvector, which IS the PageRank vector.

9. Numerical Example — 4-Node PageRank

The Transition Matrix A (from slides)

4 pages: 1, 2, 3, 4. Matrix entry = probability of going from column page to row page.

	Page 1	Page 2	Page 3	Page 4
--	--------	--------	--------	--------

Page 1	0	0	1	1/2
Page 2	1/3	0	0	0
Page 3	1/3	1/2	0	1/2
Page 4	1/3	1/2	0	0

Power Iteration Steps

Initial vector $v = [0.25, 0.25, 0.25, 0.25]$ (all equal, since $N=4$)

Iteration	Page 1	Page 2	Page 3	Page 4
v (start)	0.25	0.25	0.25	0.25
A^1v	0.37	0.08	0.33	0.20
A^2v	0.43	0.12	0.27	0.16
A^3v	0.35	0.14	0.29	0.20
A^4v	0.39	0.11	0.29	0.19
A^5v	0.39	0.13	0.28	0.19
A^6v	0.38	0.13	0.29	0.19
A^7v	0.38	0.12	0.29	0.19
A^7v ✓ CONVERGED	0.38	0.12	0.29	0.19

Final PageRank Scores (Converged at iteration 7–8):

- Page 1 = **0.38** ← Most Important (most pages link to it)
- Page 3 = **0.29**
- Page 4 = **0.19**
- Page 2 = **0.12** ← Least Important

10. Advantages & Disadvantages of PageRank

■ Advantages	■ Disadvantages
Robust against spam — hard to fake inlinks from important pages	Favors older pages — new pages haven't accumulated links yet
Query independent — computed ONCE at indexing time, works for ALL queries	Works for ALL queries

PART 2: HITS Algorithm

1. What is HITS?

HITS stands for **Hyperlink Induced Topic Search**, developed by **Jon Kleinberg** (different from PageRank which was Larry Page).

It analyzes links but in a different way than PageRank — it gives **TWO scores** to every page instead of one.

Two Types of Pages

Hub: A page that points to many good pages. Think of it like a directory or a 'best resources' blog post. Example: A page listing 'Top 10 Car Manufacturer Websites'.

Authority: A page that is pointed to by many good hubs. It has the actual valuable content. Example: Ferrari's official website, Ford's official website.

The Beautiful Relationship

- A good **Hub** points to good **Authorities**
- A good **Authority** is pointed to by good **Hubs**

They depend on each other — this creates a reinforcing loop!

Real Example From Slides

Query: 'Top automobile makers'

- **Hubs** = Car Rankings sites (edmunds.com, consumerreports.org)
- **Authorities** = Ferrari, Fiat, Ford, Honda, Chrysler official pages

2. HITS Algorithm Steps

Step 1: Given query Q, collect the **top 200 webpages** with highest frequency of that query term.

Step 2: Add pages that point to OR are pointed to by those 200 pages. Create **Adjacency Matrix A** among all these pages.

Step 3: Initialize — set **all Hub and Authority scores to 1** for every page.

Step 4: Iterate for k iterations:

- a) Update Authority scores
- b) Update Hub scores
- c) Normalize both scores

Step 5: Rank pages according to their **Authority score**.

Q: How do we build the Adjacency Matrix?

A: Simply look at which pages link to which and put 1 or 0. If Page A links to Page B → put 1. If no link → put 0. Row = page you are coming FROM. Column = page you are going TO.

Q: Till when do we iterate?

A: Same concept as PageRank — we stop when scores stop changing between iterations (convergence). In practice the slides say we run it for a fixed k number of iterations.

3. Hub & Authority Update Formulas

Authority Update Rule

$\text{auth}(p)$ = sum of Hub scores of all pages that point TO p

$$\text{auth}(p) = \text{SUM of } \text{hub}(i) \text{ for all } i \text{ that point to } p$$

Meaning: Look at ALL pages pointing to page p → add up ALL their hub scores → that sum = p's new authority score.

Hub Update Rule

$\text{hub}(p)$ = sum of Authority scores of all pages that p points TO

$$\text{hub}(p) = \text{SUM of } \text{auth}(i) \text{ for all } i \text{ that } p \text{ links to}$$

Meaning: Look at ALL pages that p links to → add up ALL their authority scores → that sum = p's new hub score.

In Matrix Form (from slides)

$$\text{Authority: } a(k) = A_{\text{transpose}} \cdot h(k-1)$$

$$\text{Hub: } h(k) = A \cdot a(k)$$

Q: Do we use normalized scores in next iterations?

A: YES. The correct flow each iteration is: (1) Calculate raw Authority scores → (2) NORMALIZE immediately → (3) Use normalized scores to calculate Hub → (4) NORMALIZE Hub scores → (5) Use those normalized Hub scores in NEXT iteration. Without normalization, numbers would grow infinitely large.

Q: Does the Adjacency Matrix change after every iteration?

A: NO. The matrix NEVER changes because it just represents WHO links to WHO. Links between pages don't change during the algorithm. What changes every iteration is only the Authority scores vector and Hub scores vector. Think of it like: Matrix A = the road map (never changes). Hub and Authority vectors = the traffic on those roads (changes every iteration).

4. Complete Numerical Example — HITS (Wikipedia, Google, Bing...)

The 6 Pages and Adjacency Matrix A

	Wiki	Google	Bing	Yahoo	Altavista	Rediff
Wikipedia	0	1	1	0	0	0
Google	1	0	1	1	1	1
Bing	0	1	0	0	0	0
Yahoo	0	0	1	0	1	0
Altavista	0	1	1	0	0	0
Rediff	0	0	1	0	0	0

1 = there IS a link | 0 = there is NO link

Step 1: Initialize

$h(0) = [1, 1, 1, 1, 1, 1]$ (all hub scores = 1)

$a(0) = [1, 1, 1, 1, 1, 1]$ (all authority scores = 1)

Step 2: First Authority Update $a(1) = A_{\text{transpose}} \cdot h(0)$

We multiply the transpose of A with the hub vector. Each page's authority = sum of hub scores of pages pointing TO it.

Counting who points to each page from the matrix:

- **Wikipedia:** Google points to it → auth = 1
- **Google:** Wikipedia, Bing, Altavista point to it → auth = 3
- **Bing:** Wikipedia, Google, Yahoo, Altavista, Rediff point to it → auth = 5
- **Yahoo:** Google points to it → auth = 1
- **Altavista:** Google, Yahoo point to it → auth = 2
- **Rediff:** Google points to it → auth = 1

Raw $a(1) = [1, 3, 5, 1, 2, 1]$

Step 3: Normalize Authority Scores

$\sqrt{1^2 + 3^2 + 5^2 + 1^2 + 2^2 + 1^2} = \sqrt{1+9+25+1+4+1} = \sqrt{41} = 6.403$

Page	Raw Score	Calculation	Normalized Score
Wikipedia	1	$1 / \sqrt{41}$	0.15617
Google	3	$3 / \sqrt{41}$	0.46852
Bing	5	$5 / \sqrt{41}$	0.78087 ← HIGHEST
Yahoo	1	$1 / \sqrt{41}$	0.15617
Altavista	2	$2 / \sqrt{41}$	0.31235
Rediff	1	$1 / \sqrt{41}$	0.15617

Step 4: Hub Update $h(1) = A \cdot a(1)$ [using raw scores 1,3,5,1,2,1]

Each page's hub score = sum of authority scores of pages it links TO.

Wikipedia links to Google(3), Bing(5):

$= (0 \times 1) + (1 \times 3) + (1 \times 5) + (0 \times 1) + (0 \times 2) + (0 \times 1) = 0 + 3 + 5 + 0 + 0 + 0 = 8$

Google links to Wiki(1), Bing(5), Yahoo(1), Altavista(2), Rediff(1):

$= (1 \times 1) + (0 \times 3) + (1 \times 5) + (1 \times 1) + (1 \times 2) + (1 \times 1) = 1 + 0 + 5 + 1 + 2 + 1 = 10$

Bing links to Google(3) only:

$= (0 \times 1) + (1 \times 3) + (0 \times 5) + (0 \times 1) + (0 \times 2) + (0 \times 1) = 0 + 3 + 0 + 0 + 0 + 0 = 3$

Yahoo links to Bing(5), Altavista(2):

$= (0 \times 1) + (0 \times 3) + (1 \times 5) + (0 \times 1) + (1 \times 2) + (0 \times 1) = 0 + 0 + 5 + 0 + 2 + 0 = 7$

Altavista links to Google(3), Bing(5):

$= (0 \times 1) + (1 \times 3) + (1 \times 5) + (0 \times 1) + (0 \times 2) + (0 \times 1) = 0 + 3 + 5 + 0 + 0 + 0 = 8$

Rediff links to Bing(5) only:

$$= (0 \times 1) + (0 \times 3) + (1 \times 5) + (0 \times 1) + (0 \times 2) + (0 \times 1) = 0 + 0 + 5 + 0 + 0 + 0 = 5$$

Raw $h(1) = [8, 10, 3, 7, 8, 5]$

Step 5: Normalize Hub Scores

$$\text{sqrt}(8^2 + 10^2 + 3^2 + 7^2 + 8^2 + 5^2) = \text{sqrt}(64 + 100 + 9 + 49 + 64 + 25) = \text{sqrt}(311) = 17.635$$

Page	Raw Score	Calculation	Normalized Score
Wikipedia	8	8 / 17.635	0.4538
Google	10	10 / 17.635	0.5673 ← BEST HUB
Bing	3	3 / 17.635	0.1702
Yahoo	7	7 / 17.635	0.3970
Altavista	8	8 / 17.635	0.4538
Rediff	5	5 / 17.635	0.2836

Iteration 1 Conclusion:

- **Bing** is the strongest Authority (most pages link to it) — $\text{auth} = 0.78087$
- **Google** is the best Hub (links to most authoritative pages) — $\text{hub} = 0.5673$
- These normalized scores are then used as input for Iteration 2, and so on until convergence.

5. HITS Issues (From Slides)

Issue 1 — Query Dependent: HITS scores change with every query. Different search term = completely different Hub and Authority scores. Unlike PageRank which has one fixed score for every page.

Issue 2 — Slow Performance: Because it is query dependent, it must run at query time. Every time someone searches, HITS runs the entire algorithm — much slower than PageRank.

Issue 3 — Not Commonly Used: Most search engines don't use HITS. Only notable use was Teoma (bought by Ask Jeeves/Ask.com). PageRank became the industry standard.

Issue 4 — Two Scores Per Page: HITS computes Hub AND Authority for every page — more complex than PageRank's single score.

Issue 5 — Works on Small Subset: HITS only works on the top 200 relevant pages (focused subgraph). PageRank works on the entire web.

6. Final Comparison: HITS vs PageRank

Feature	HITS	PageRank
Scores per page	2 (Hub + Authority)	1 (PageRank score)
When executed	At query time (slow)	At indexing time (fast)
Spam resistance	Not robust against spam	Robust against spam
Page age bias	Never favors old pages	Favors older pages
Query dependency	Query DEPENDENT	Query INDEPENDENT
Coverage	Small subset (~200 pages)	Entire web
Common usage	Rarely used (Teoma)	Widely used (Google)

Quick Summary:

- **PageRank** = calculated once, fast at search time, but biases old pages
- **HITS** = calculated fresh every query, more flexible, but slower and easier to spam